THE UNIVERSITY OF DODOMA

COLLEGE OF INFORMATICS AND VIRTUAL EDUCATION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GROUP ASSIGNMENT REPORT

GROUP NUMBER: 07

MODULE NAME: MICROCONTROLLER SYSTEMS

MODULE CODE: CT 321

SEMESTER: TWO

ACADEMIC YEAR: 2025 / 2026

PBL TITLE: EXTERNAL INTERRUPT-BASED EMERGENCY STOP AND EVENT COUNTER SYSTEM

PARTICIPANTS

S/N	NAME	REG NUMBER	PROGRAMME
1	EZEKIEL LEMANA	T23-03-00241	CE3
2	HARUN KADILO	T23-03-01251	CE3
3	JACKSON MANDA	T23-03-18278	CE3
4	YASSIN JABIR	T23-03-14605	CE3

1. Introduction

Many embedded systems must react immediately to external events such as emergency stop buttons, object detection sensors, alarms, or motion triggers. A polling-based program may miss short input pulses if the processor is busy with another task. External interrupts solve this problem by allowing the microcontroller to stop normal execution temporarily and run a service routine when an important event occurs.

This project demonstrates how ATmega32 external interrupts can be used to build a safe event-driven system. The system counts external event pulses in normal mode, enters emergency-stop mode when the emergency button is pressed, prevents counting during emergency mode, and returns to normal operation using a recovery/reset button.

2. Problem Statement and Objectives

The problem is to design a controller that can count normal events while still responding immediately to an emergency condition. The emergency input must dominate the normal counting operation so that the system stops counting and activates alarm indicators without delay.

2.1 Objectives

- To design an ATmega32-based event counter using external interrupts.
- To implement immediate emergency-stop response using INT0.
- To count event pulses during normal mode using INT1.
- To recover from emergency mode and reset the count using INT2.
- To indicate system state using green LED, red LED, buzzer, and UART serial output.
- To design the schematic and PCB layout in KiCad and generate Gerber files.
- To verify the system using SimulIDE screenshots and a demonstration video.

3. Expected System Operation

Input / Event	Expected Response
System powered ON	Green LED ON, red LED OFF, buzzer OFF, serial shows NORMAL mode with count 0.
Object/event pulse detected	Event count increases only when the system is in normal mode.
Emergency stop button pressed	System enters EMERGENCY STOP mode; red LED and buzzer turn ON; counting stops.
Event pulse during emergency	Event count does not increase.
Recovery button pressed during emergency	System returns to NORMAL mode and keeps the current count.
Recovery/reset button pressed in normal mode	Event count resets to zero.

4. System Design

4.1 System Block Diagram

The block diagram shows the external interrupt inputs, ATmega32 controller, output indicators, buzzer, serial terminal, and 5 V power supply used in the system.

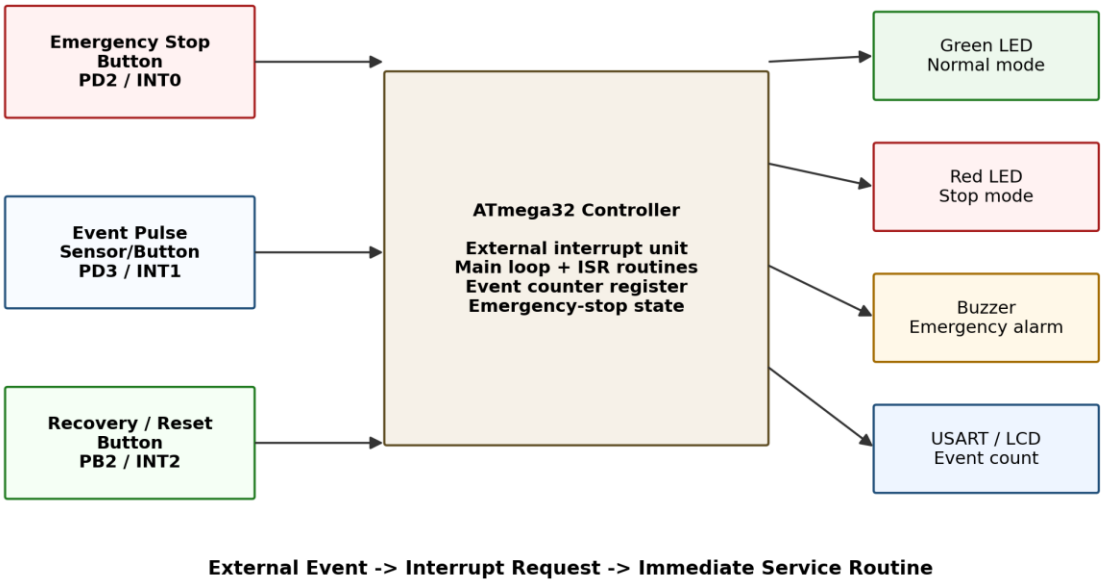


Figure 1: System block diagram.

4.2 Components Used

Component	Quantity	Purpose
ATmega32 / ATmega32A	1	Main microcontroller and interrupt controller.
Emergency stop push button	1	Triggers INT0 and starts emergency mode.
Event pulse button / sensor	1	Triggers INT1 and increments the event count.
Recovery/reset push button	1	Triggers INT2; recovers from emergency or resets count.
MCU reset button	1	Hardware reset for the ATmega32.
Green LED	1	Normal mode indicator.
Red LED	1	Emergency stop indicator.
Active buzzer	1	Emergency alarm output.
NPN transistor and 1 kOhm resistor	1 set	Buzzer driver stage.
330 Ohm resistors	2	LED current limiting.
10 kOhm resistors	4	RESET and interrupt input pull-up resistors.
8 MHz crystal and 22 pF capacitors	1 set	External clock circuit.
100 nF capacitors	2	VCC and AVCC decoupling.
3-pin serial header	1	+5V, serial TX, and GND connection.

4.3 ATmega32 Pin Mapping

ATmega32 Pin	DIP Pin	Direction	Net / Function	Connected Device
PB0	1	Output	LED_NORMAL	R2 330 Ohm -> Green LED -> GND
PB1	2	Output	LED_STOP	R3 330 Ohm -> Red LED -> GND
PB2 / INT2	3	Input	RECOVERY_INT2	SW3 -> GND, R7 10 kOhm -> +5V
PB3	4	Output	BUZZER	R4 1 kOhm -> Q1 base -> Buzzer driver
RESET	9	Input	RESET	R1 10 kOhm -> +5V, reset button -> GND
VCC	10	Power	+5V	Digital supply and decoupling
GND	11	Power	GND	Ground zone
XTAL1	13	Input	XTAL1	8 MHz crystal and C3 22 pF
XTAL2	14	Output	XTAL2	8 MHz crystal and C4 22 pF
PD1 / TXD	15	Output	SERIAL_TX	J1 pin 2, UART 9600 baud
PD2 / INT0	16	Input	EMERGENCY_INT0	SW1 -> GND, R5 10 kOhm -> +5V
PD3 / INT1	17	Input	EVENT_INT1	SW2 -> GND, R6 10 kOhm -> +5V
AVCC	30	Power	+5V	Analog/digital supply and decoupling
GND	31	Power	GND	Ground zone

5. Interrupt and Firmware Design

5.1 Interrupt Configuration

The input buttons are wired as active-LOW inputs. The inputs are normally HIGH through pull-up resistors and become LOW when a button is pressed. Therefore, falling-edge triggering is used for INT0, INT1, and INT2.

Interrupt	MCU Pin	Trigger	Firmware Action
INT0	PD2	Falling edge	Set emergency_mode = 1; red LED and buzzer ON; counting locked.
INT1	PD3	Falling edge	Increment event_count only if emergency_mode == 0.
INT2	PB2	Falling edge	If emergency, recover to normal; if normal, reset count to 0.

5.2 Firmware Flowchart

The firmware initializes GPIO, UART, Timer0, and the three external interrupts. The main loop updates outputs and serial messages, while the ISRs handle urgent events.

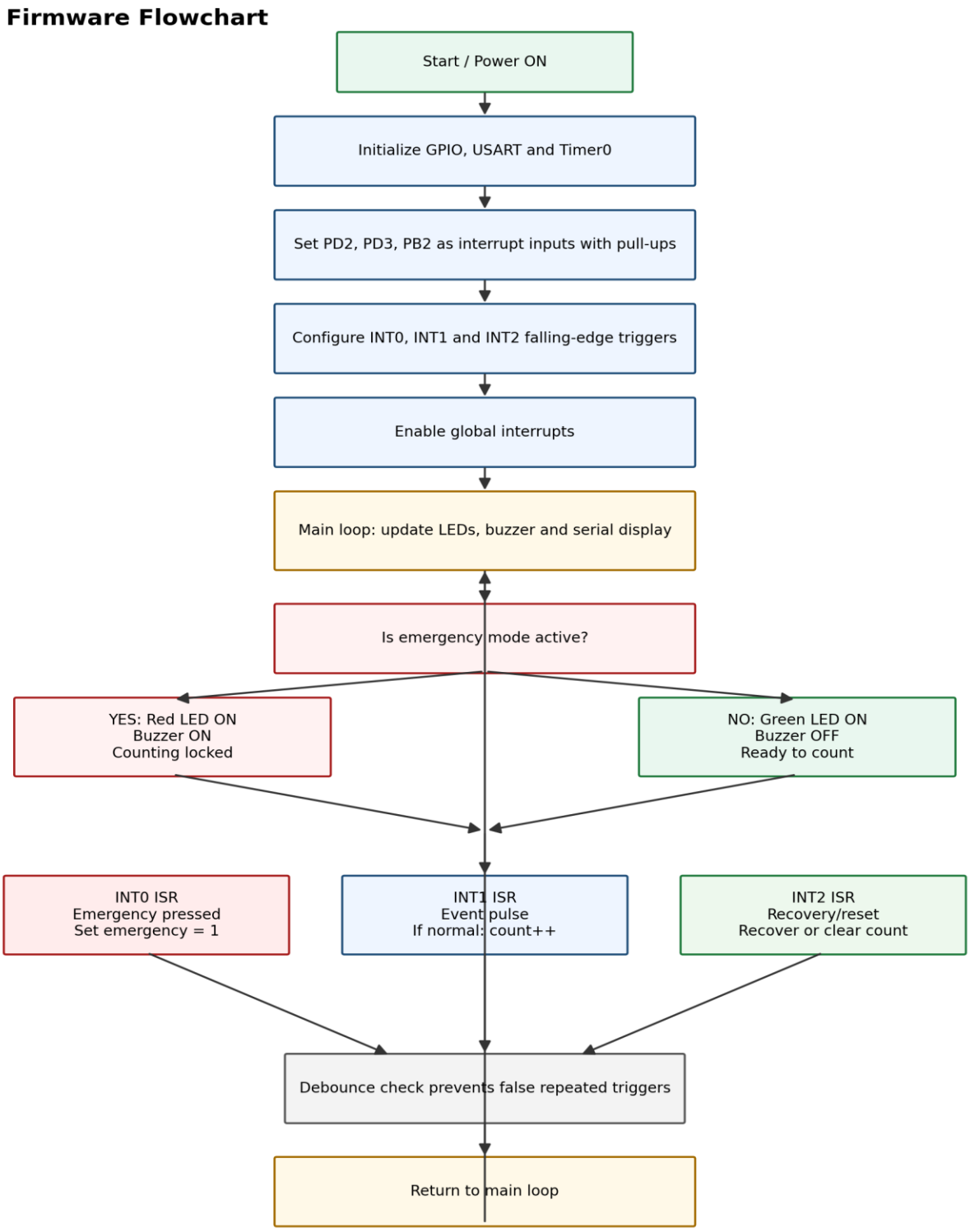


Figure 2: Firmware flowchart.

5.3 Key Firmware Logic

Timer0 generates a 1 ms system tick. A 200 ms debounce window is applied separately to each external interrupt to prevent false multiple triggers caused by mechanical button bounce.

```
#define DEBOUNCE_MS   200UL
#define HEARTBEAT_MS 1000UL

INT0: emergency stop
INT1: event count only in normal mode
INT2: recovery/reset
```

5.4 Interrupt Service Routine Summary

```
ISR(INT0_vect)  -> emergency_mode = 1;
ISR(INT1_vect)  -> if (!emergency_mode) event_count++;
ISR(INT2_vect)  -> if emergency: emergency_mode = 0; else: event_count = 0;
```

The final compiled HEX file used in simulation is `firmware/hex/emergency_event_counter.hex`. The verified build used an 8 MHz clock and produced approximately 1403 bytes of flash use and 80 bytes of SRAM use.

6. KiCad Schematic and PCB Design

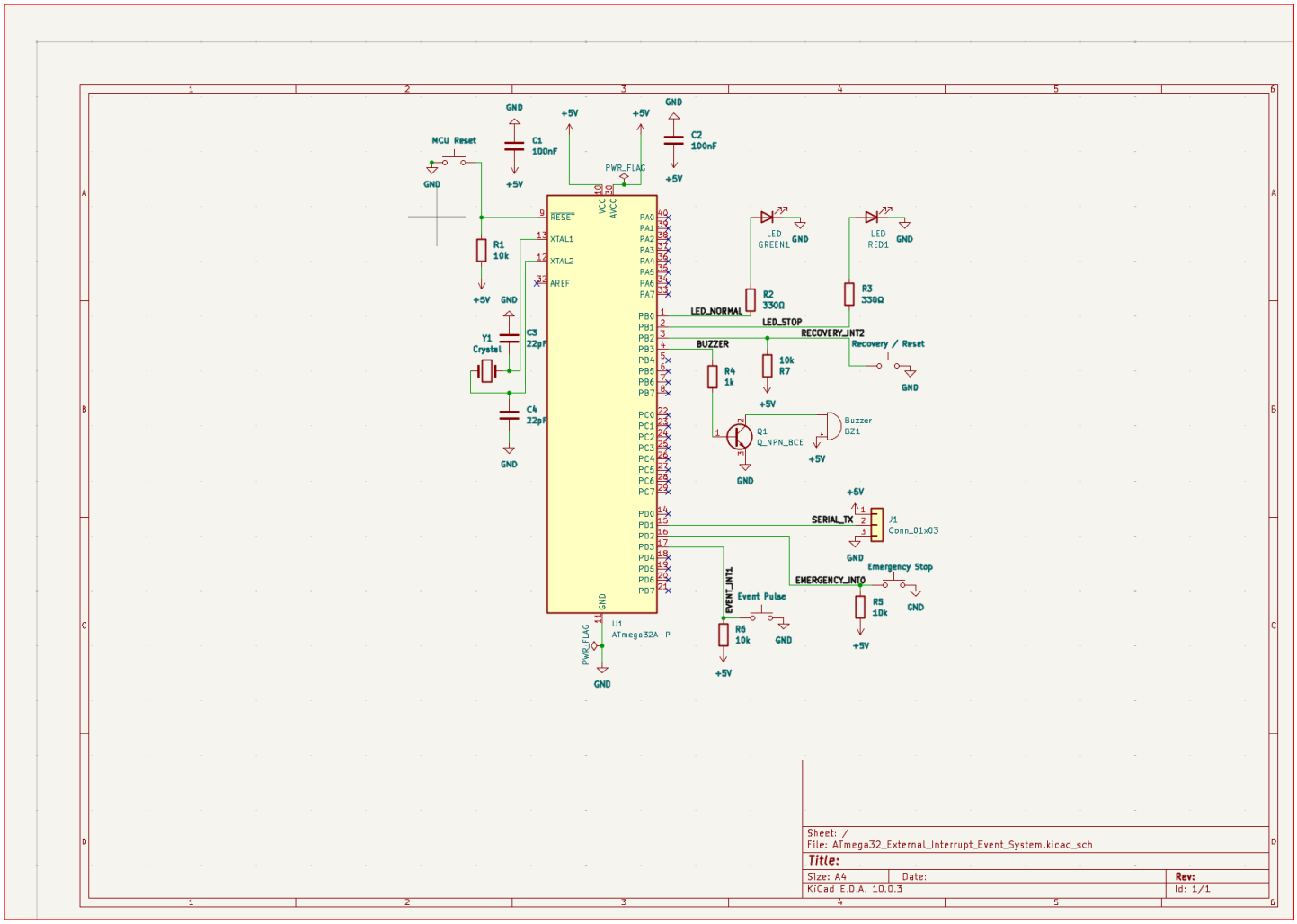


Figure 3: Final KiCad schematic.

PCB Layout

The PCB layout was routed using KiCad. A B.Cu copper zone was used for GND and an F.Cu copper zone was used for +5V. Interrupt input traces, output traces, and the serial header are labelled through the schematic net names.

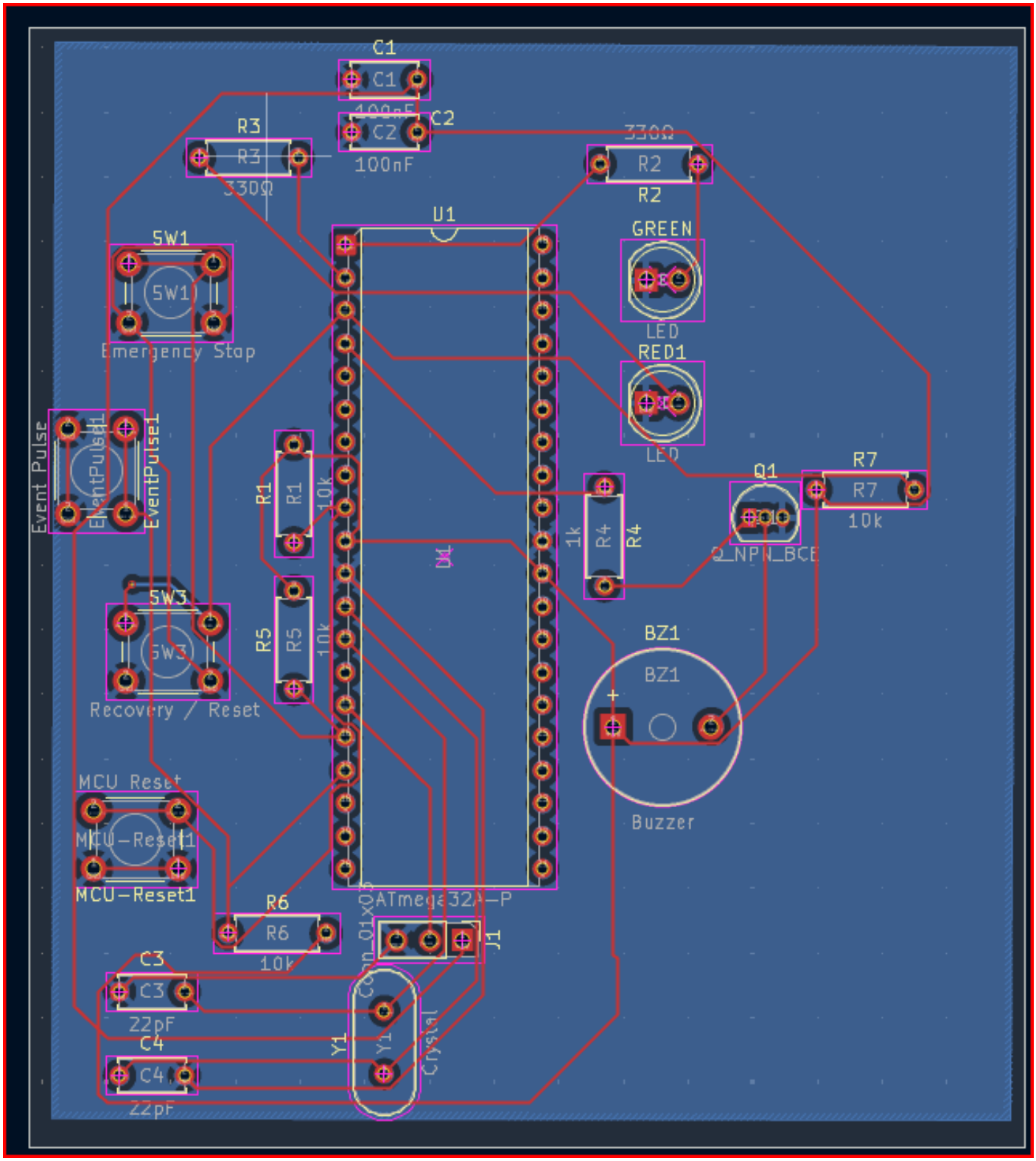


Figure 4: Final KiCad PCB layout.

Design Rule Check and Gerber Output

The final PCB passed KiCad DRC with zero violations, zero unconnected items, zero errors, and zero warnings. Gerber and drill files were exported and stored in `kicad/gerber/`.

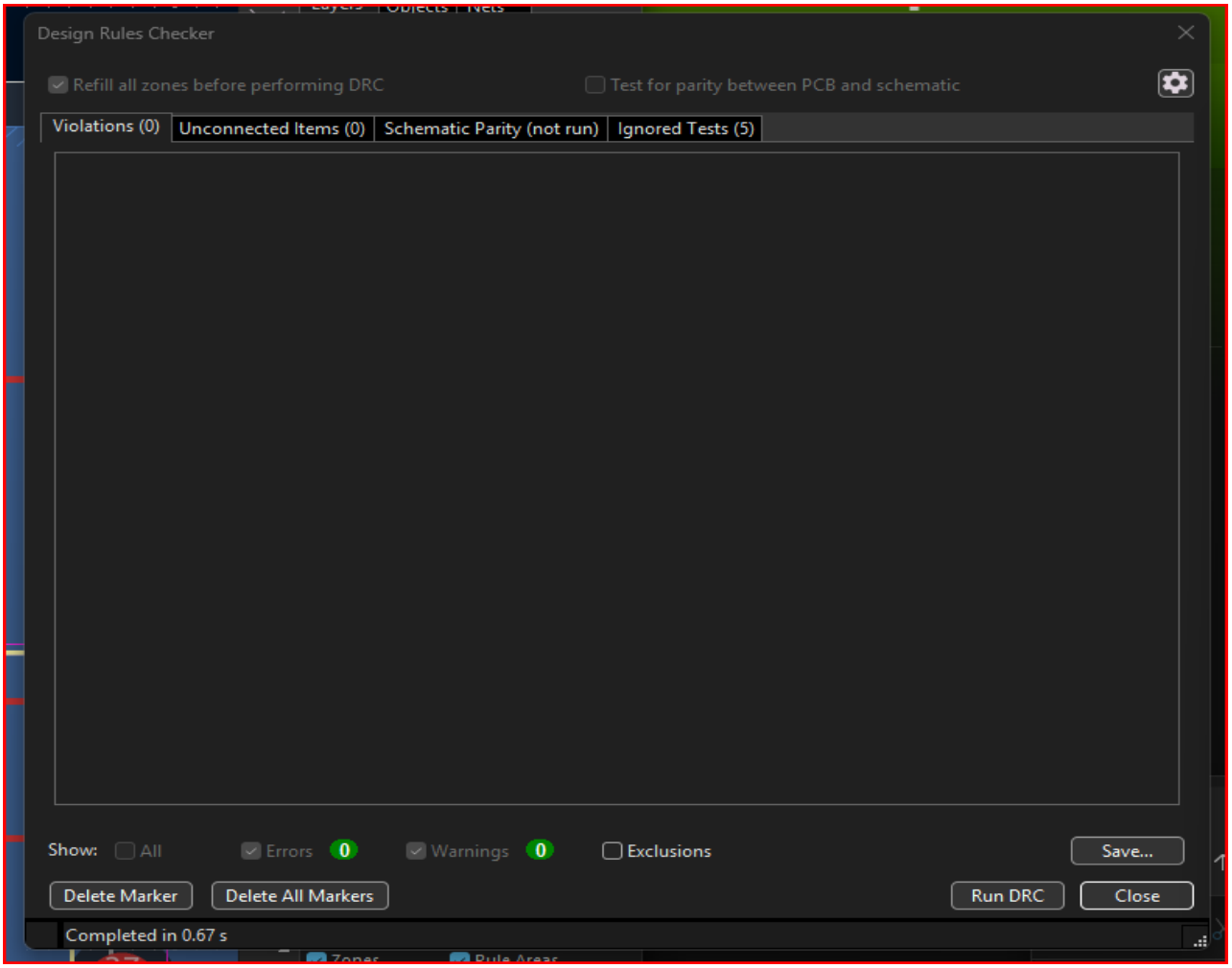
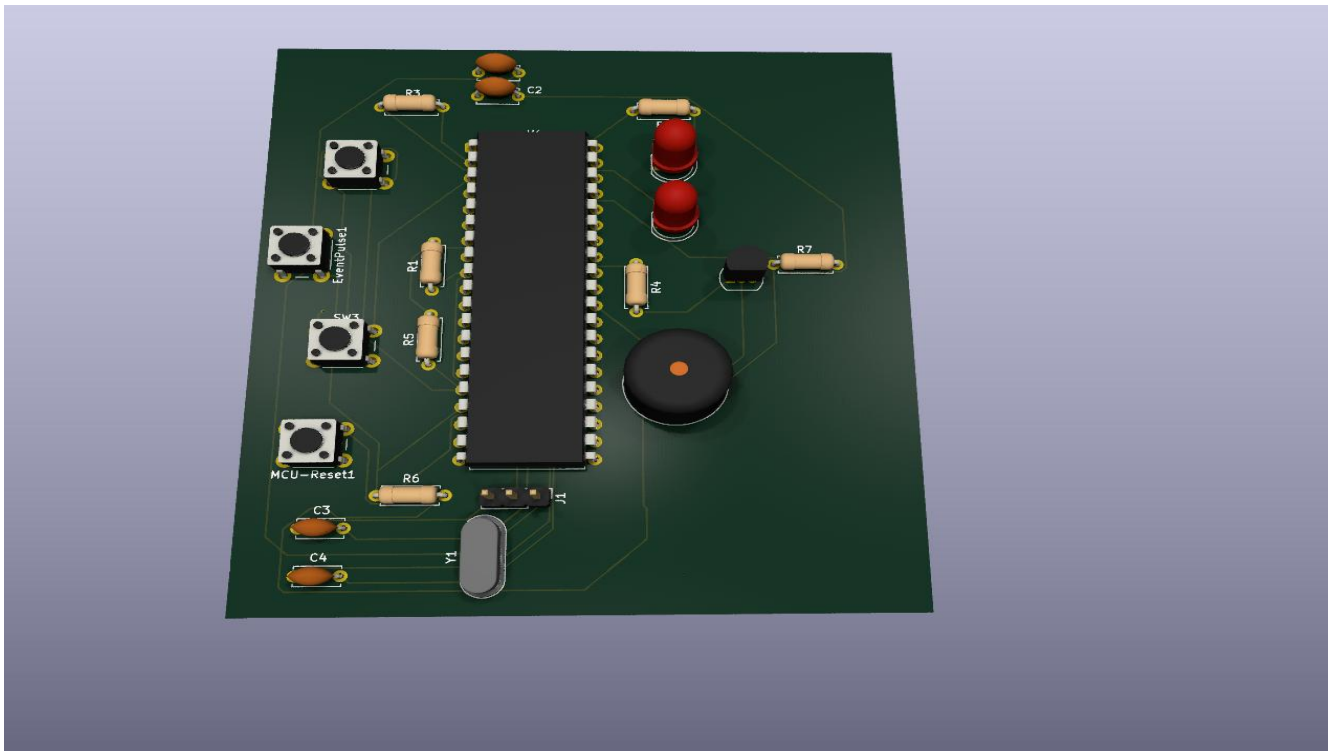
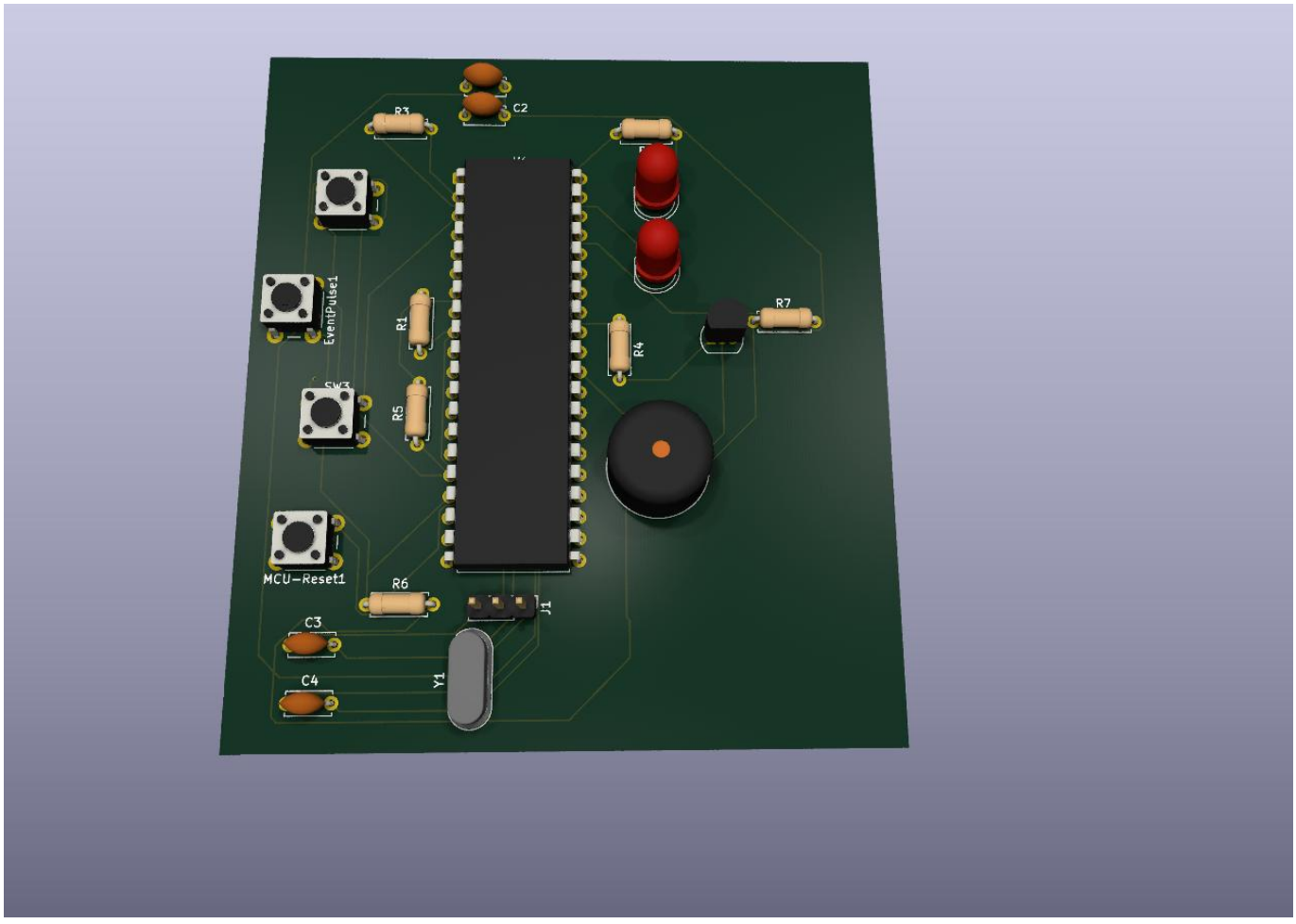


Figure 5: KiCad DRC result showing zero violations and zero unconnected items.

Gerber / Drill Output	Location
Front copper	kicad/gerber/*-F_Cu.gbr
Back copper	kicad/gerber/*-B_Cu.gbr
Solder mask and silkscreen	kicad/gerber/*-Mask.gbr and *-Silkscreen.gbr
Board outline	kicad/gerber/*-Edge_Cuts.gbr
Drill files	kicad/gerber/*-PTH.drl and *-NPTH.drl
Gerber zip	kicad/gerber/ATmega32_Gerber_Files.zip

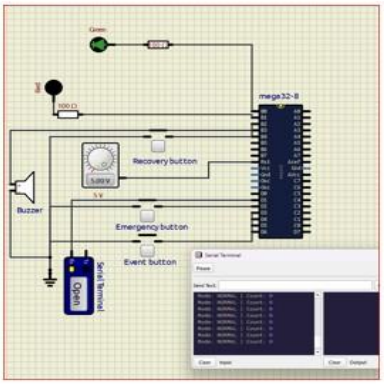
PCB 3D-VIEW


R3
C2
EventPulse1
R1
R4
R7
SW3
R5
MCU-Reset1
R6
J1
C3
C4
Y1

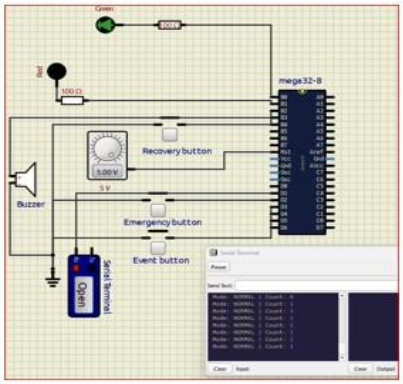

R3
C2
EventPulse1
R1
R4
R7
SW3
R5
MCU-Reset1
R6
J1
C3
C4
Y1

7. Simulation Screenshot Summary

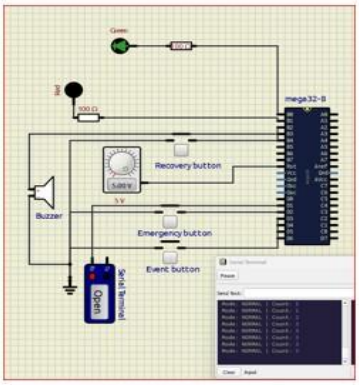
01_normal_startup.png



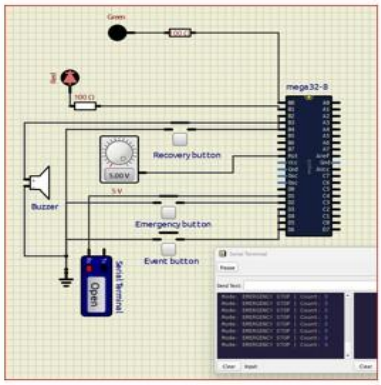
02_single_event_count.png



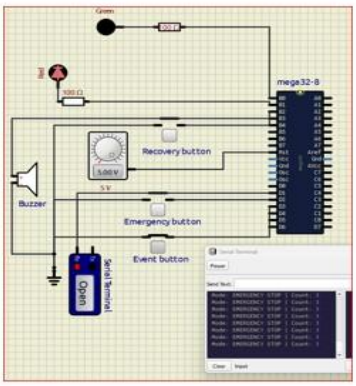
03_multiple_event_count.png



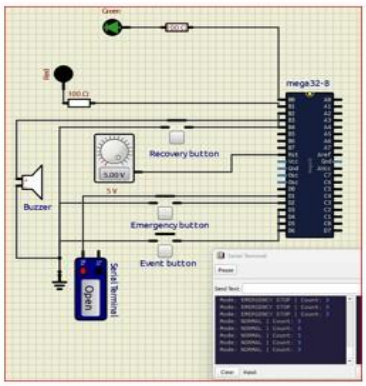
04_emergency_stop.png



05_counting_lockout.png



06_recovery_mode.png



07_reset_normal_mode.png

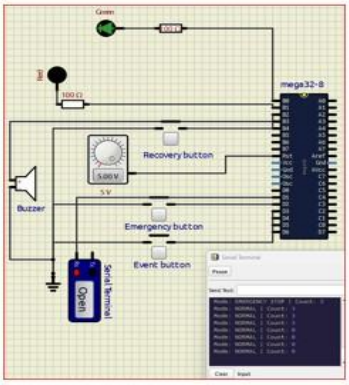


Figure 6: Contact sheet of all SimulIDE test screenshots.

8. Test Results

No.	Test Case	Expected Output	Observed Output	Result
1	Normal startup	Green LED ON, red LED OFF, buzzer OFF, count 0.	System starts in NORMAL mode and serial output shows count 0.	PASS
2	Single event pulse	One event pulse increments count to 1.	Serial output shows NORMAL mode with count 1.	PASS
3	Multiple event pulses	Repeated events increase count step by step.	Count increases correctly after multiple event button presses.	PASS
4	Emergency stop	Red LED and buzzer ON; counting stops.	System enters EMERGENCY STOP mode and alarm output is activated.	PASS
5	Event during emergency	Count must not increase.	Count remains unchanged while emergency mode is active.	PASS
6	Recovery mode	System returns to NORMAL mode and preserves count.	Serial output returns to NORMAL mode with previous count retained.	PASS
7	Reset in normal mode	Pressing recovery/reset in normal mode resets count to 0.	Serial output shows NORMAL mode with count 0.	PASS

8.1 Individual Simulation Evidence

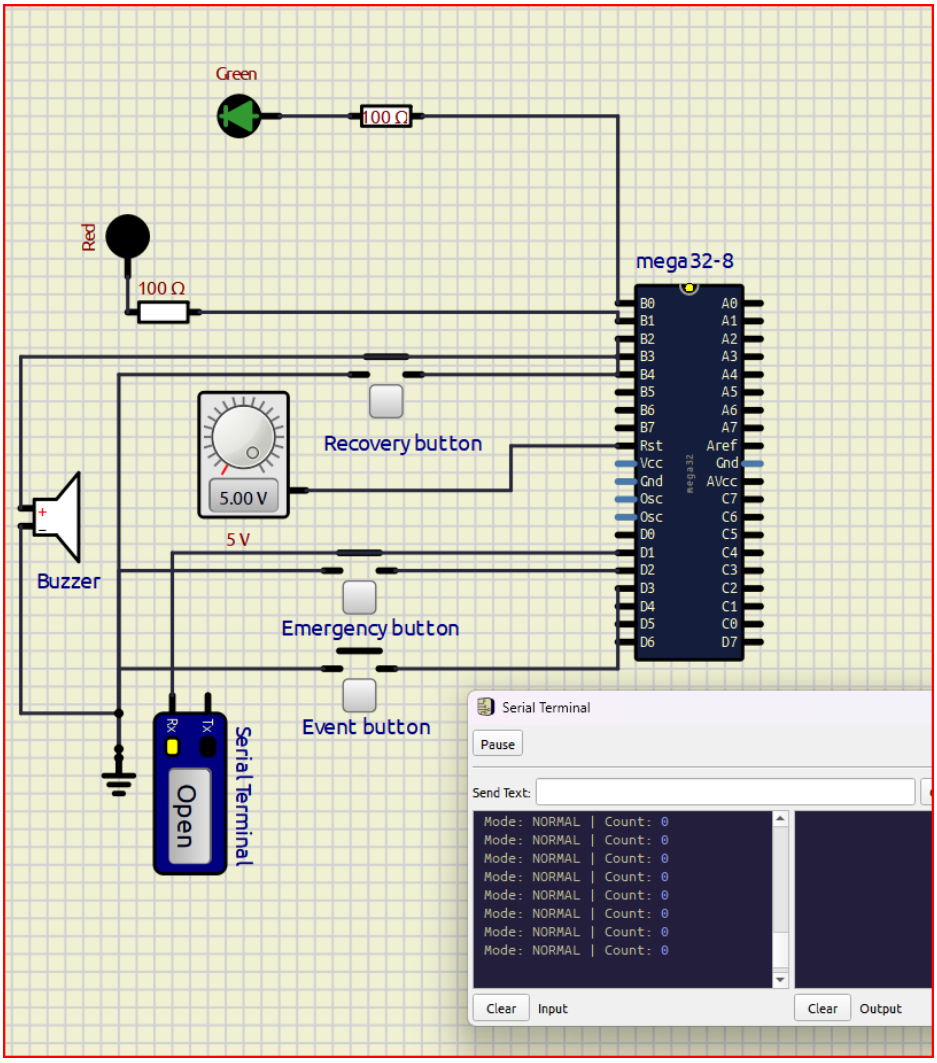


Figure 7: Normal startup - green LED ON and count 0.

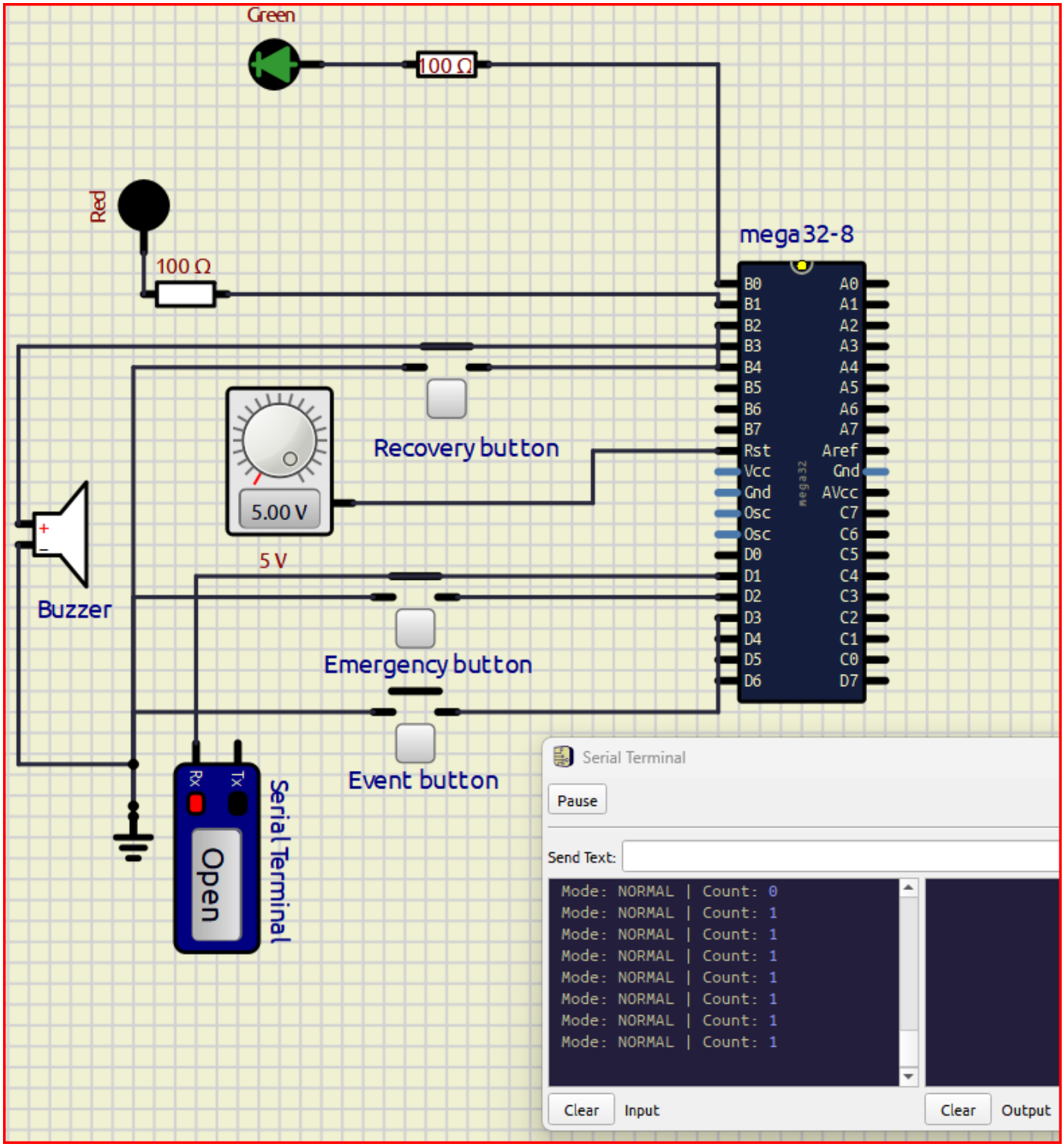


Figure 8: Single event pulse - count increases to 1.

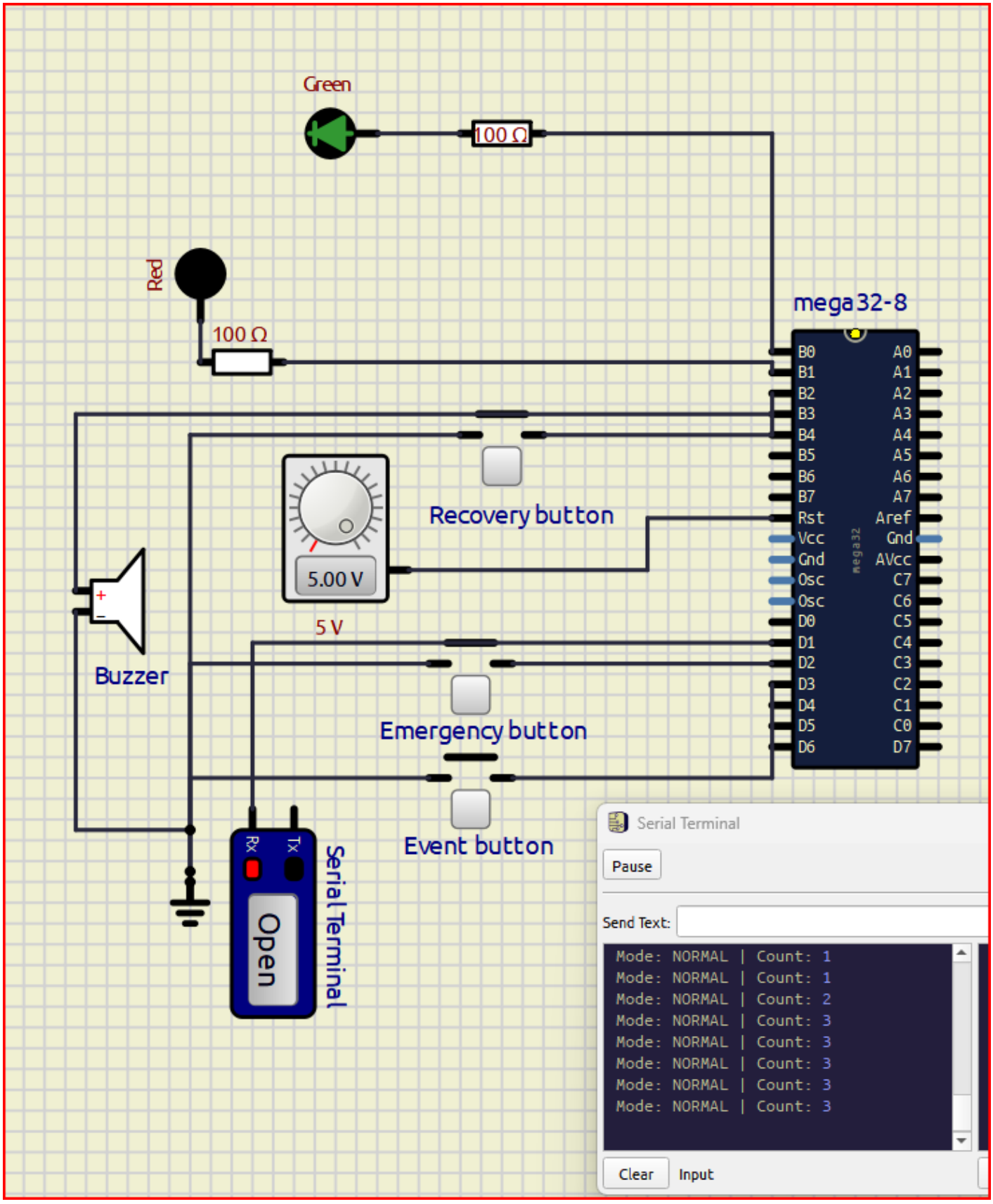


Figure 9: Multiple event pulses - count increases correctly.

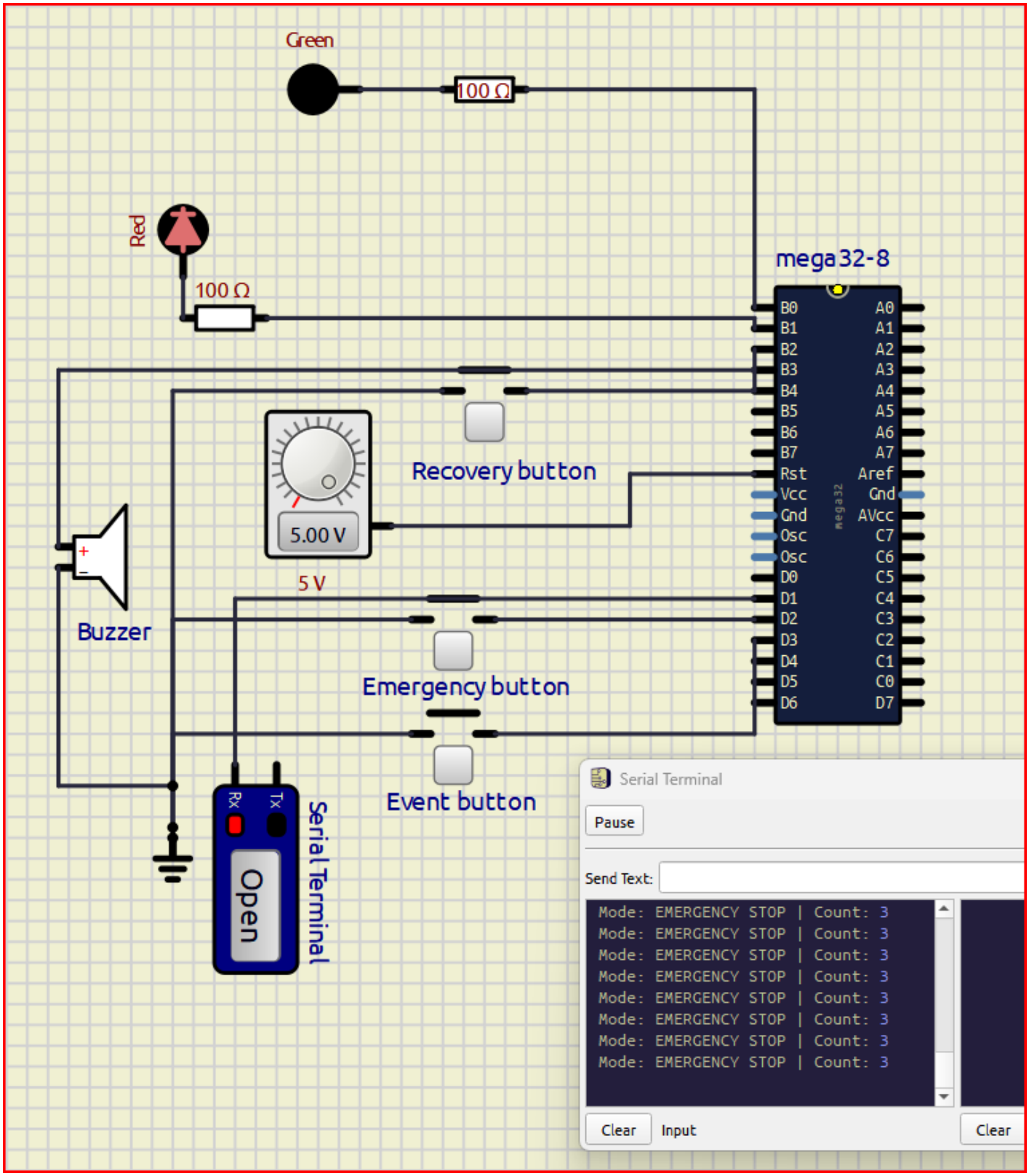


Figure 10: Emergency stop - emergency mode active.

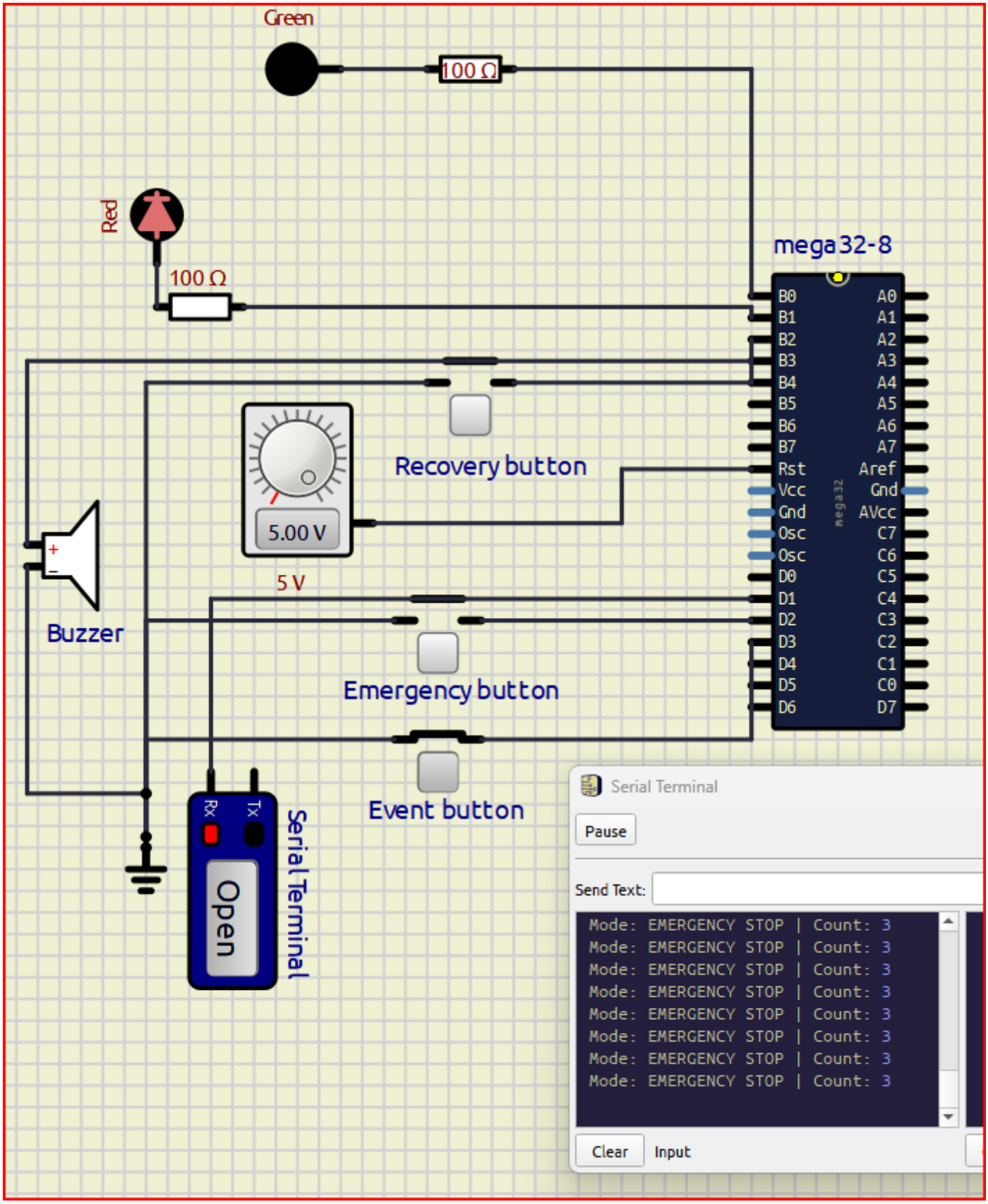


Figure 11: Event during emergency - count remains locked.

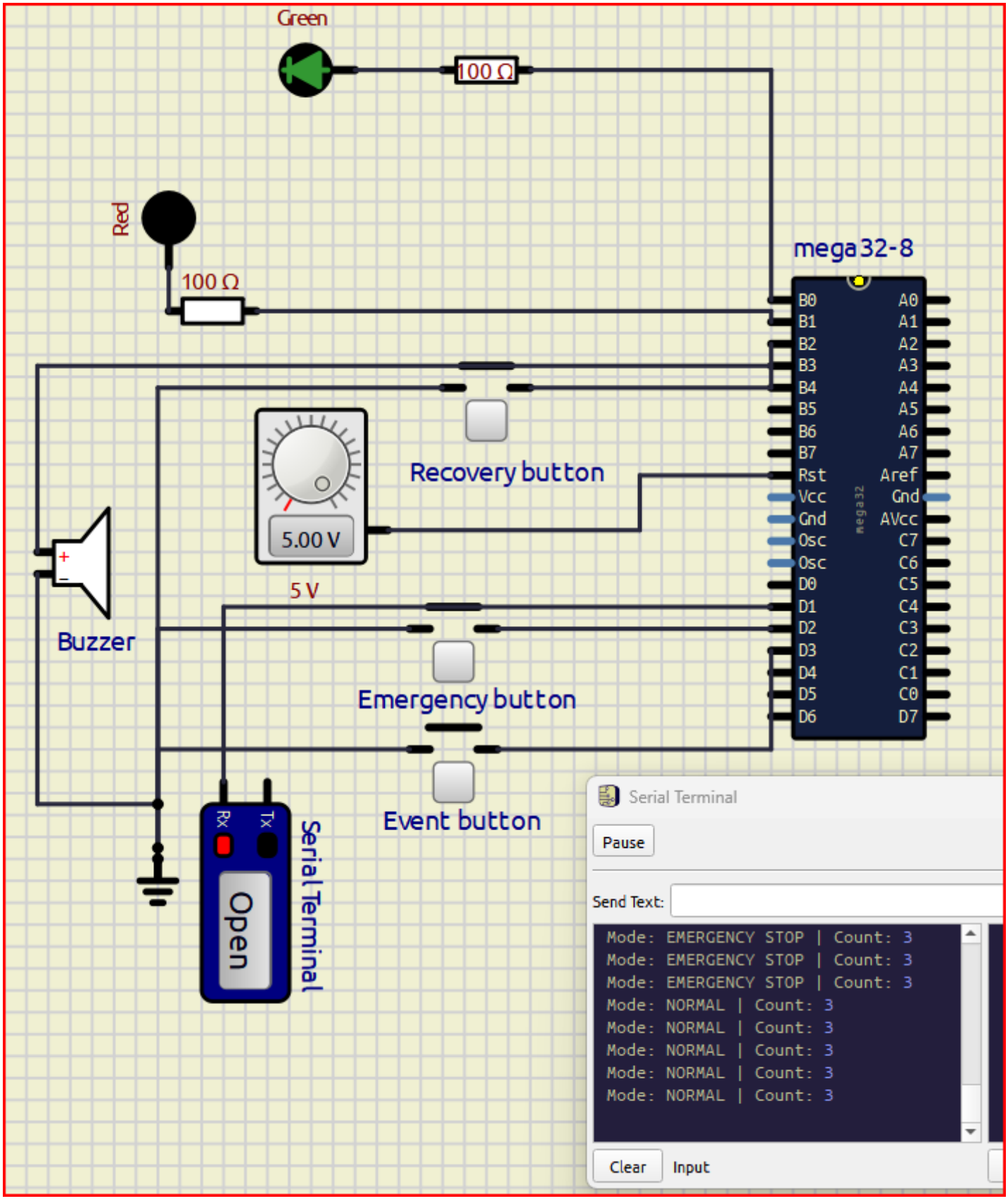


Figure 12: Recovery mode - system returns to normal mode.

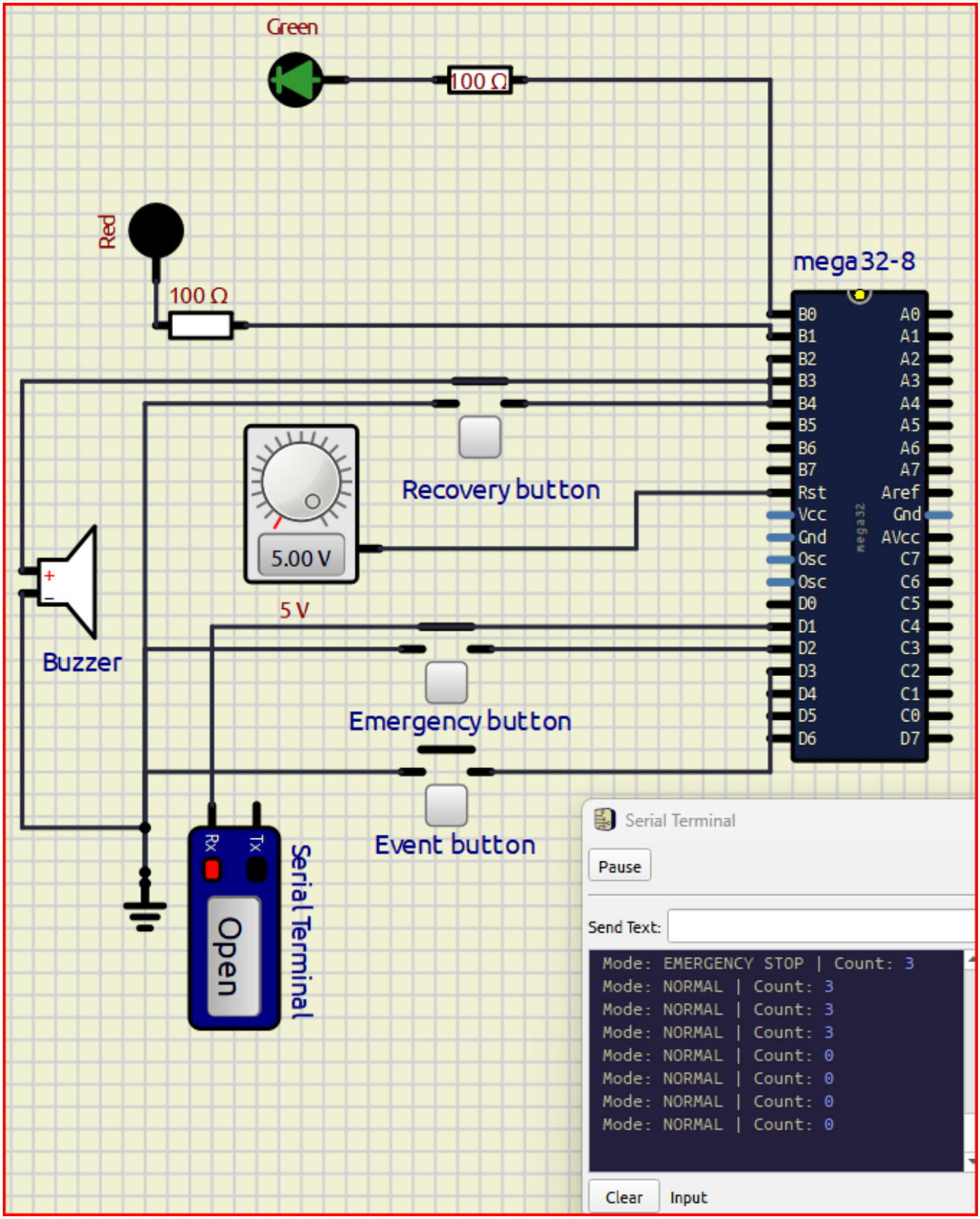


Figure 13: Reset in normal mode - count resets to 0.

9. Challenges and Solutions

Challenge	Solution
RESET pin issue in SimulIDE	The RESET pin was held HIGH using a 5 V source so the firmware could run normally.
Mechanical button bounce	Timer0 was used to create a 1 ms tick and each interrupt line uses a 200 ms debounce window.
Many GND and +5V connections on the PCB	Copper zones were used: B.Cu for GND and F.Cu for +5V.
PCB routing unconnected items	KiCad DRC was used to locate and fix each missing connection until DRC showed 0 violations.
Emergency and reset button behavior	INT2 was designed with dual behavior: recover during emergency, reset count when already in normal mode.

10. Repository and File Organization

```
ATmega32_External_Interrupt_Event_System/
├── README.md
├── firmware/
│   ├── src/main.c
│   └── hex/emergency_event_counter.hex
├── kicad/
│   ├── ATmega32_External_Interrupt_Event_System/
│   ├── schematic/
│   ├── pcb/
│   └── gerber/
├── simulide/
│   ├── circuit/emergency_event_counter.sim1
│   └── screenshots/
├── documentation/
│   ├── block_diagram.png
│   ├── firmware_flowchart.png
│   ├── interrupt_configuration.md
│   ├── pin_mapping_table.md
│   ├── test_results.md
│   └── final_report/
└── media/
    └── demo_video.mp4
```

11. Conclusion

The project successfully demonstrates an ATmega32 external interrupt system for emergency stop and event counting. INT0 handles the emergency stop, INT1 handles event pulse counting, and INT2 handles recovery/reset. The system correctly locks event counting during emergency mode and returns to normal operation after recovery. The implementation also includes debounce handling, UART status display, LED and buzzer outputs, KiCad schematic and PCB design, Gerber export, SimulIDE screenshots, and a demonstration video.